

Software Requirements and Design Document

For

Group Flappy Bird Clone

Version 1.0

Authors:

Kameryn G

Luke G

Jake S

Andrew B

Gabriel M

1. Overview (5 points)

Give a general overview of the system in 1-2 paragraphs (similar to the one in the project proposal).

2. Functional Requirements (10 points)

List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.

3. Non-functional Requirements (10 points)

List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.

4. Use Case Diagram (10 points)

This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.

Textual descriptions of use cases: For the first increment, the textual descriptions for the use cases are not required. However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.

5. Class Diagram and/or Sequence Diagrams (15 points)

This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.

If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system**.

If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams, but for all the use cases of your system**. In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.

Class Diagrams show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.

A **Sequence Diagram** simply depicts **interaction between objects** (or **functions** - in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.

6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

Overview:

The Flappy Bird Clone is a 2D arcade-style game developed in C++ using the SFML graphics library. The application replicates the core mechanics of the classic Flappy Bird title while introducing enhanced gameplay elements, including a player health tracking system, temporary invincibility frames (I-frames) upon taking damage, and continuous score progression. Players navigate a bird through procedurally generated pipe obstacles by controlling its vertical velocity in real time.

Designed for cross-platform desktop compatibility, the game features an interactive main menu, collision detection algorithms, and real-time physics loops. The primary objective is to maximize the score by successfully passing through pipe gaps while managing player health across continuous obstacles.

Functional Requirements:

- The system will allow the user to control the vertical velocity of the bird by pressing the space bar.
- The system will continuously generate pipe obstacles of varying height as the screen moves from left to right.
- The system will use AABB collision for the bird, pipe, ground, and the sky.
- The system will track the health and current score of the bird as it progresses through the level while enabling I-frames after colliding and incrementing the score by one for every pipe gap passed through.

Non-Functional Requirements:

- Performance: The gameplay loop will maintain a consistent 60 fps to ensure constant physics and visual updates.
- Portability: This software operates on standard x86-64 hardware. It has been created to run on Windows, macOS, and Linux operating systems using standard C++ and CMake without source modifications.

Use Case Diagram:

Actor: Player

Use Case 1: Start Game - The player selects the start option from the main menu. The system initializes the GameState, resetting bird position, player health, score, and pipe vectors before transitioning into the active gameplay state.

Use Case 2: Flap the Bird - While in the active playing state, the player presses the spacebar. The system detects the input event and applies upward velocity (BIRD_JUMP_VEL) to the bird object to counteract gravity.

Use Case 3: Menu Navigation - The player navigates game menus using keyboard or mouse inputs to start a new game, pause, or exit the application.

Class/Sequence Diagram:

Class-

Structs:

GameState - GameState status, bool alive, int health, int score, sf::Clock clock, sf::Time lastTick, sf::Time lastHitTime, sf::Time lastPipeSpawn, std::vector<Pipe> pipes, Bird bird / resetGame(GameState&), runTickLogic(GameState&, sf::Window&)

Bird - sf::Vector2f pos, float yVel

Pipe - float xPos, float gapPos, float gapHeight, bool passed

Sequence-

1. Physics Tick

- main() calls runTickLogic().
- runTickLogic() calculates delta time (dt) and updates Bird.yVel (gravity) and Bird.pos.
- runTickLogic() iterates through the pipes vector, decrementing xPos.
- runTickLogic() calls checkCollisions().
- checkCollisions() evaluates checkCircleAABBCollision() for the bird and nearest pipe.
- If true, GameState health is decremented.

2. User Input

main() loops through window.pollEvent().
Player presses the spacebar triggering sf::Event::KeyPressed.
System checks GameState.status.
If Playing and alive, the system sets Bird.yVel = BIRD_JUMP_VEL.

Operating Environment:

- This software operates on standard x86-64 hardware. It has been created to run on Windows, macOS, and Linux operating systems.

Assumptions/Dependencies:

- Assume the user is operating with a standard keyboard and mouse while operating on a machine that graphics system hardware supports OpenGL.